

Sprawozdanie: Implementacja algorytmu Diffiego-Hellmana uzgadniania klucza sesyjnego

Jakub Biernat 160248

1. Kod aplikacji:

```
import sys
from sympy import factorint
import math
import random

def is_prime(a):
    if a < 2:
        return False

    for i in range(2, int(math.sqrt(a)) + 1):
        if a % i == 0:
            return False

    return True

def generate_prime(bits):
    while True:
        n = random.getrandbits(bits)
        n |= (1 << bits - 1) | 1
        if is_prime(n):
            return n

def is_primitive_root(g, n):
    factors = factorint(n - 1)
    for factors in factors:
        if pow(g, (n - 1) // factors, n) == 1:
            return False
    return True

def generate_max_primitive_root(n):
    for g in range(n - 1, 1, -1):
        if is_primitive_root(g, n):
            return g
    return None

def main():
    n = 0
    g = None
    while g is None:
        n = generate_prime(int(sys.argv[1]))
        g = generate_max_primitive_root(n)
    print("Krok 1.")
    print("A i B: wybrane n = " + str(n))
    print("A i B: wybrane g = " + str(g))
    x = random.randint(10000, 99999)
    print("Krok 2.")
    print("A: wybrany klucz prywatny x = " + str(x))
    X = pow(g, x, n)
```

```

print("A: obliczone X = " + str(X))
print("Krok 3.")
y = random.randint(10000, 99999)
print("B: wybrany klucz prywatny y = " + str(y))
Y = pow(g, y, n)
print("B: obliczone Y = " + str(Y))
print("Krok 4.")
print("A i B: wysyłają do siebie obliczone X i Y")
print("Krok 5.")
kA = pow(Y, x, n)
print("A: obliczone k = " + str(kA))
print("Krok 6.")
kB = pow(X, y, n)
print("B: obliczone k = " + str(kB))
print("Krok 7.")
if kA == kB:
    print("Wygenerowany klucz sesji: " + str(kA))
else:
    print("Niezgodność wygenerowanych kluczy")

main()

```

2. Ograniczenia dla użytych parametrów

Liczba n powinna być dużą liczbą pierwszą, do testów używałem liczba z zakresu 32-64 bity, liczby x i y również powinny być dużymi liczbami całkowitymi, w testach użyłem losowych wartości z przedziału $<10000; 99999>$,

3. Jakie dane można podsłuchać i czy możesz zaproponować jakiś schemat ataku?

Można podsłuchać n , g , X oraz Y . Jednym ze sposobów ataku może być sprawdzanie wszystkich wartości x w celu odszyfrowania wiadomości, jednak jest to wydajne jedynie przy małym zakresie liczb. Innym sposobem jest taktyka Man in the Middle, poprzez “wpięcie się” w komunikację między użytkownikami:

- I. A wysyła X do B -> atakujący przechwytuje
- II. Atakujący wysyła własne X do B
- III. B wysyła Y -> atakujący przechwytuje
- IV. Atakujący wysyła własne Y do A
- V. Teraz przy przesyłaniu wiadomości atakujący może je odszyfrowywać, zmieniać oraz ponownie szyfrować

4. Dodatkowe wnioski

Zaimplementowany algorytm działa poprawnie, jednak jest podatny na szybkie złamanie klucza przy małych wartościach n , oraz brak uwierzytelniania użytkowników, co grozi podszywaniem się pod innych, przez co w praktyce konieczne jest zastosowanie dodatkowych zabezpieczeń.